

Password Strength Analyzer

A Password Strength Analyzer checks how secure a password is by evaluating:

- Length
- Character variety
- Predictability
- Reuse risk
- Common attack patterns

Key Features

- Check password length, complexity, and uniqueness
- Suggest stronger password alternatives
- Optional: Integrate with a database to prevent reuse of old passwords

Core Security Rules

Requirement	Recommendation
Length	12–16+ characters
Uppercase	At least 1
Lowercase	At least 1
Numbers	At least 1
Symbols	At least 1
Uniqueness	Avoid reused passwords
Common words	Avoid dictionary words

Password Entropy Concept

Password strength is often measured using entropy.

$$H = L \times \log_2(N)$$

Where:

H = entropy (strength in bits)

L = password length

N = size of character set used

Higher entropy means stronger passwords.

Example Python Implementation

```
import re
import math

COMMON_PASSWORDS = [
```

```

    "password", "123456", "qwerty", "admin", "welcome"
]

def calculate_entropy(password):
    charset = 0

    if re.search(r"[a-z]", password):
        charset += 26
    if re.search(r"[A-Z]", password):
        charset += 26
    if re.search(r"[0-9]", password):
        charset += 10
    if re.search(r"^[A-Za-z0-9]", password):
        charset += 32

    entropy = len(password) * math.log2(charset) if charset else 0
    return round(entropy, 2)

def analyze_password(password):
    score = 0
    feedback = []

    # Length check
    if len(password) >= 12:
        score += 2
    elif len(password) >= 8:
        score += 1
    else:
        feedback.append("Password is too short.")

    # Complexity checks
    if re.search(r"[A-Z]", password):
        score += 1
    else:
        feedback.append("Add uppercase letters.")

    if re.search(r"[a-z]", password):
        score += 1
    else:
        feedback.append("Add lowercase letters.")

    if re.search(r"[0-9]", password):
        score += 1
    else:
        feedback.append("Add numbers.")

    if re.search(r"^[A-Za-z0-9]", password):
        score += 1
    else:
        feedback.append("Add special characters.")

    # Common password check
    if password.lower() in COMMON_PASSWORDS:
        feedback.append("Password is too common.")
        score = 0

    entropy = calculate_entropy(password)

    # Strength rating
    if score <= 2:
        strength = "Weak"
    elif score <= 5:
        strength = "Moderate"
    else:
        strength = "Strong"

    return {
        "strength": strength,
        "entropy": entropy,
        "feedback": feedback
    }

# Example usage
password = input("Enter password: ")

result = analyze_password(password)

print("\n\nPassword Strength:", result["strength"])
print("Entropy:", result["entropy"], "bits")

```

```

if result["feedback"]:
    print("\n\nSuggestions:")
    for item in result["feedback"]:
        print("-", item)

```

Example Output

```

Enter password: Hello123

Password Strength: Moderate
Entropy: 52.44 bits

Suggestions:
- Add special characters.

```

Strong Password Suggestions

```

import secrets
import string

def generate_password(length=16):
    chars = (
        string.ascii_letters +
        string.digits +
        string.punctuation
    )

    return ''.join(secrets.choice(chars) for _ in range(length))

print(generate_password())

Why use secrets instead of random?
• secrets is cryptographically secure
• Better for authentication systems
• Resistant to predictable generation

```

Optional Advanced Features

```

1. Prevent Password Reuse

Store hashed old passwords in a database.

Never store plaintext passwords.

Use hashing algorithms like:
• bcrypt
• argon2
• scrypt

import bcrypt

password = b"MySecurePassword123!"

hashed = bcrypt.hashpw(password, bcrypt.gensalt())

# Verify later
bcrypt.checkpw(password, hashed)

```

Detect Breached Passwords

```

You can integrate with:
Have I Been Pwned API

```

`https://haveibeenpwned.com/API/v3`

This checks whether a password appears in known data breaches.

Security Best Practices

- Never log passwords
- Use HTTPS for transmission
- Enforce minimum entropy
- Add rate limiting
- Use MFA/2FA
- Salt all password hashes
- Avoid storing security answers in plaintext

Expected Learning Outcomes

By building this project, you will learn:

- Password security principles
- Entropy and randomness
- Regular expressions
- Secure hashing
- Cryptographic best practices
- Input validation
- Authentication fundamentals